

3주차 · Gazebo VRX 구축과 좌표계

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

- 과목: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- 이번 주차 학습 내용: ① 시뮬레이터 설치 ② 배를 물에 띄우고 직접 조종 ③ 좌표계 정리

★ 시작 전 확인

- 2주차 ROS 2 설치가 끝나 있어야 함
- `ros2 run demo_nodes_cpp talker` 가 되는지 먼저 확인할 것
- 저장공간 20 GB 이상: `df -h` 로 확인
- 빌드에 30~60분 걸림. 충전기를 꽂고, 절전 모드로 두지 말 것

이 주차를 마치면 할 수 있어야 하는 것

1. SDF · URDF · Xacro 의 역할 차이 설명
2. Gazebo Garden + VRX 설치하고 WAM-V를 물에 띄우기
3. 명령어로 배를 직진 · 선회시키기
4. 선박 6자유도와 ENU / NED / Body 좌표계 구분
5. 쿼터니언 ↔ 오일러각 변환 이해
6. TF2 트리 읽기

준비물

항목	내용
환경	2주차에 만든 ROS 2 Humble (<code>ros2 topic list</code> 가 동작해야 함)
저장공간	20 GB 이상 — Gazebo Garden + VRX 빌드에 필요
전원	충전기 지참. VRX 빌드가 30~60분 걸린다
그래픽	GUI 가 뜨는지 1주차 2-4절로 미리 확인

✕ 이번 주차에는 빌드 시간이 길다

`colcon build` 를 걸어 놓고 그동안 1부 이론(좌표계)을 듣는 순서로 진행한다.
수업 시작하자마자 빌드를 먼저 건다.

1부 · 이론

1-1. 물리 시뮬레이터는 무엇을 계산하는가

매 시간 스텝마다 반복되는 일

1. 외력 계산
 - 추진기 플러그인 -> 추력
 - 유체력 플러그인 -> 부가질량, 감쇠, 부력
 - 파랑 플러그인 -> 파력
 - 바람 플러그인 -> 풍력
2. 강체 동역학 적분
 - 힘 = 질량 x 가속도 -> 새로운 위치 · 자세 · 속도
3. 센서 시뮬레이션
 - GNSS : 참값 + 잡음
 - IMU : 참값 + 바이어스 + 잡음
 - LiDAR : 레이캐스팅 -> 포인트클라우드
 - 카메라: 렌더링 -> 영상
4. ROS 2 토픽으로 발행

- 기본 스텝: 1 ms

! 본 제어기는 이 루프의 밖에 있음

- Simulink나 Python 노드가 토픽으로 상태를 받아 명령을 돌려줌
- Gazebo가 그것을 다음 스텝의 추력으로 사용
- 즉 **협조 시뮬레이션**이지, 한 스텝씩 맞물려 도는 방식이 아님

시뮬레이션 시간과 RTF

용어	뜻
sim time	시뮬레이터 안의 시간. 벽시계 시간과 다름
RTF (Real Time Factor)	시뮬레이션 시간 ÷ 실제 시간

- RTF = 1.0 → 실시간과 같은 속도
- VRX + 무거운 센서 → 보통 **0.3 ~ 0.7**
- Gazebo 창 하단에 표시됨

⚠ 알고리즘의 오류가 아니라 처리 속도 문제일 수 있다

노트북 성능이 낮으면 RTF가 떨어짐. **항상 RTF를 먼저 확인할 것.**

1-2. SDF · URDF · Xacro

형식	정체	쓰임
SDF	Gazebo 전용 XML	월드 전체 — 바다, 부표, 도킹 스테이션, 조명
URDF	ROS 표준 로봇 기술 XML	로봇 한 대 — 링크, 조인트, 센서
Xacro	URDF를 만들어 내는 매크로	반복 제거, 값 대입

왜 Xacro가 필요한가

- 추진기 · 센서는 위치만 다르고 구조는 같음
- Xacro 없이 쓰면 같은 XML 블록을 개수만큼 복사해야 함
- 실제 파일 `vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_aft_thrusters.xacro`
 - 본 과목에서 쓰는 기본 구성이다

```
<xacro:include filename="$(find wamv_description)/urdf/thrusters/engine.xacro" />
<xacro:engine prefix="left" position="-2.373776 1.027135 0.318237" />
<xacro:engine prefix="right" position="-2.373776 -1.027135 0.318237" />
```

- `xacro:engine` 매크로 하나를 위치만 바꿔 두 번 호출
- 매크로 없이 쓰면 같은 XML 블록을 두 번 복사해야 함
- 추진기가 4개, 6개로 늘어나면 차이가 커짐

★ 본 과목은 후방 2추진기 기본 모델을 쓴다

- 좌 · 우 각 1개, 선미($x = -2.374$ m)에 좌우로 ± 1.027 m 떨어져 있음
- 장착각을 지정하지 않았으므로 **선수 방향으로 고정**
- 좌우 추력 차이로 방향을 바꾸는 **차동 추진(differential thrust)**
- 3주차 실습에서 이미 이렇게 조종해 봤음

- 변환 흐름

```
component_config.yaml └─
thruster_config       └─> Xacro 처리 -> URDF -> SDF -> Gazebo에 스폰
wamv_gazebo.urdf.xacro └─
```

- 실제 파일은 4주차에 열어 봄
 - `src/vrx/vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_aft_thrusters.xacro`

1-3. VRX 플러그인 — 무엇이 배를 움직이는가

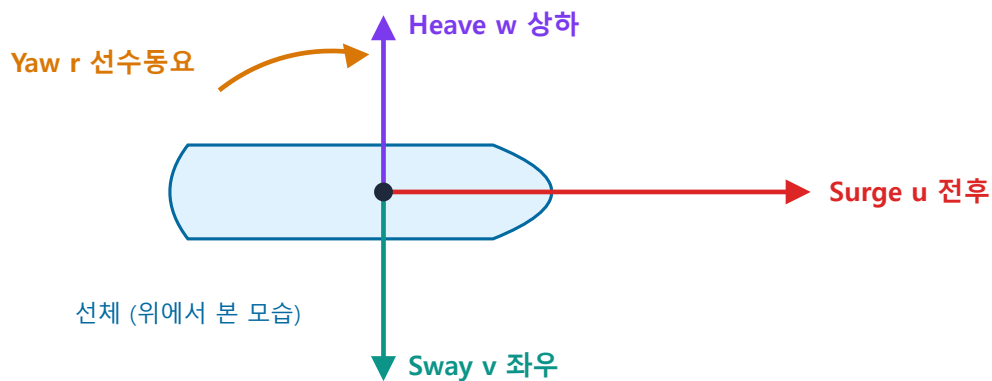
플러그인	역할	조절 가능한 것
Hydrodynamics	부가질량, 감쇠	유체력 계수
Buoyancy	부력	배수량, 흘수
Wavefield	파랑 생성·파력	파고, 주기, 방향
Wind	풍력	풍속, 방향
Thruster	추력 생성	추력계수, 최대 명령
Joint Position Controller	추진기 틸트각 (본 과목에서는 0 고정, 미사용)	각도 명령
센서	측정값 + 잡음	갱신주기, 잡음 크기

! 왜 알아야 하는가

14주차에 "파랑 3단계, 바람 3단계" 조건으로 반복시험을 함.
그때 조절하는 것이 바로 이 파라미터들임.

1-4. 선박의 6자유도 운동

선박의 6자유도 운동 - 이 수업은 이 중 3개만 제어한다



제어 대상 3자유도 : Surge · Sway · Yaw

파랑 진동으로 보고 제외 : Heave · Roll · Pitch

자유도	기호	속도	이름	뜻
전후	x	u	Surge	앞뒤로 나아감
좌우	y	v	Sway	옆으로 미끄러짐
상하	z	w	Heave	위아래로 오르내림
횡경사	φ	p	Roll	좌우로 기울어짐
종경사	θ	q	Pitch	앞뒤로 기울어짐
선수각	ψ	r	Yaw	뱃머리 방향이 돌아감

3자유도 근사

- 수상선의 수평면 운동은 보통 **Surge · Sway · Yaw** 3자유도로 다룸

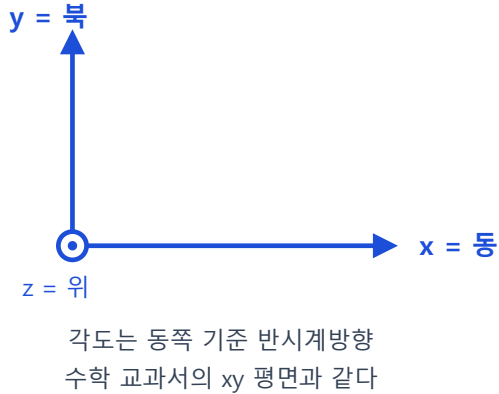
$\eta = [x, y, \psi]$ 위치와 선수각 (지구 고정 좌표계, NED)
 $v = [u, v, r]$ 속도 (선체 고정 좌표계, Body)

- Heave · Roll · Pitch 는 **파랑에 의한 진동**으로 보고 제어 대상에서 제외
- 7주차에 Fossen 운동방정식으로 정식화함

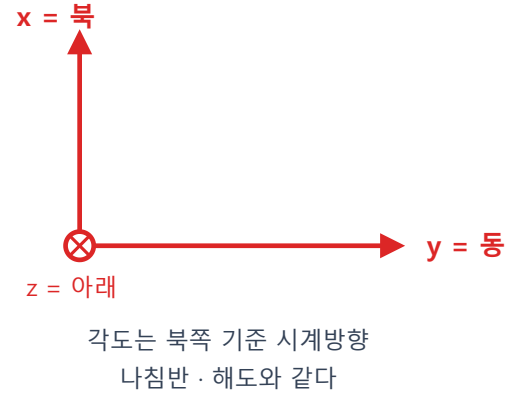
1-5. 좌표계 — 본 과목에서 가장 혼동하기 쉬운 부분

ENU 와 NED - 같은 장소를 다르게 부른다

ENU (ROS · Gazebo 가 쓰는 것)



NED (선박 · 제어 이론이 쓰는 것)



$N = y_{\text{ENU}}$

$E = x_{\text{ENU}}$

$D = -z_{\text{ENU}}$

$\psi_{\text{NED}} = 90^\circ - \psi_{\text{ENU}}$

세 개의 좌표계

좌표계	축	각도 기준	누가 쓰는가
ENU	x=동, y=북, z=위	동쪽 기준 반시계	ROS · Gazebo
NED	x=북, y=동, z=아래	북쪽 기준 시계	선박 · 항공 제어
Body	x=선수, y=우현, z=아래	—	배와 함께 움직임

× 되돌릴 수 없는 조작

- VRX(ROS/Gazebo)는 **ENU** 로 데이터를 줌
- 본 과목 제어 이론과 Simulink 모델은 **NED** 기준임
- 변환을 빠뜨리면 배가 반대로 돌거나 90도 어긋난 방향으로 감
- 그런데 **에러가 나지 않음**. 동작만 예상과 달라진일 뿐임

변환 공식

위치

$N = y_{\text{ENU}}$ (북 = ENU의 y)
 $E = x_{\text{ENU}}$ (동 = ENU의 x)
 $D = -z_{\text{ENU}}$ (아래 = ENU의 z를 뒤집음)

- 이 변환은 **자기 자신이 역변환**임. NED → ENU 도 같은 식을 씀

방위각

$\text{psi_NED} = 90^\circ - \text{psi_ENU}$ (그 뒤 $-180^\circ \sim +180^\circ$ 범위로 정리)

각속도

$r_NED = -r_ENU$ (z축 방향이 반대이므로 부호가 뒤집힘)

잘못했을 때의 증상

빠뜨린 것	증상
x/y 교환	북쪽으로 가라고 했는데 동쪽으로 감
z 부호	좌선회 명령에 우선회함
각도 변환	헤딩이 90도 어긋난 채로 그럭저럭 따라감 ← 가장 위험

⚠ 부분적으로 동작하는 상태가 가장 위험하다

근사적으로 동작하므로 지나치기 쉬우나, 조건이 바뀌면 성능이 급격히 저하됨.
그래서 이번 주차 과제는 **수치로 증명**하게 함.

위경도를 미터로 — LLA → NED

- GPS는 위도 · 경도 · 고도(LLA)를 줌
- 제어기는 미터 단위 지역 좌표가 필요
- **Flat-Earth 근사** (수 km 이내에서 유효)

$\Delta N = (\text{위도} - \text{기준위도}) \times \text{자오선 곡률반경}$
 $\Delta E = (\text{경도} - \text{기준경도}) \times \text{묘유선 곡률반경} \times \cos(\text{기준위도})$
 $\Delta D = -(\text{고도} - \text{기준고도})$

- MATLAB에서는 한 줄

```
ned = lla2ned([lat lon alt], lla0, 'flat');
```

- 본 과목의 기준점 (시드니 레가타 해역)

```
lla0 = [-33.72276870341191, 150.67399057896623, 1.183941401541233]
```

1-6. 회전 표현 — 오일러각 · 쿼터니언 · 회전행렬

① 오일러각 — 사람이 읽기 좋음

- $(\phi, \theta, \psi) = (\text{roll}, \text{pitch}, \text{yaw})$
- 선박 · 항공은 **Z-Y-X 순서** (yaw → pitch → roll)
- 장점: 직관적. "선수각 30도" 하면 바로 이해됨
- 단점: **짐벌락**

짐벌락이란

- pitch가 $\pm 90^\circ$ 가 되면 roll 축과 yaw 축이 겹침
- 자유도 하나를 잃고, 변환식의 분모가 0이 되어 값이 튼

```
roll = atan2( 2(wx + yz), 1 - 2(x^2 + y^2) )
pitch = asin ( 2(wy - zx) ) <- 이 값이 ±1에 가까워지면
yaw = atan2( 2(wz + xy), 1 - 2(y^2 + z^2) ) <- roll 과 yaw 가 불안정해짐
```

! 수상선은 pitch가 ±90도가 될 일이 없음

그러면 배가 뒤집힌 것임. 그래서 실무상 오일러각으로 다루어도 됨.
다만 **저장과 전송은 쿼터니언**이 표준이고, ROS 메시지도 전부 쿼터니언임.

② 쿼터니언 — 컴퓨터가 좋아함

- $q = (w, x, y, z)$, 크기가 1인 4개 숫자
- 장점: 짐벌락 없음, 계산 효율적
- 단점: 값만 봐서는 자세를 알 수 없음

✗ 순서 관례가 두 가지임

- ROS · Gazebo: (x, y, z, w) — **w가 마지막**
- MATLAB `quaternion` 객체: (w, x, y, z) — **w가 처음**

이 불일치로 인한 버그가 매년 나옴.

변환 코드를 짤 때 **반드시 주석으로 순서를 명시**할 것.

③ 회전행렬 (DCM)

- 3×3 행렬. 벡터를 한 좌표계에서 다른 좌표계로 옮김
- 수평면 3자유도에서 Body → NED

$$J(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$[\dot{x}, \dot{y}, \dot{\psi}] = J(\psi) \times [u, v, r]$$

정리

용도	권장 표현
저장 · 전송 (ROS 메시지)	쿼터니언
사람에게 보여주기, 로그	오일러각
벡터 좌표 변환	회전행렬

1-7. TF2 — 좌표계 관계를 관리하는 시스템

왜 필요한가

- 로봇에는 좌표계가 많음

```

map          지도 원점 (고정)
|
odom         주행거리계 원점
|
base_link    선체 중심
+-- lidar_link    LiDAR 장착 위치
+-- camera_link   카메라 장착 위치
+-- gps_link
+-- imu_link

```

- TF2가 푸는 문제
 - "LiDAR가 자기 앞 10 m에서 부표를 봤다. 그 부표는 지도상 어디인가?"

정적 TF vs 동적 TF

종류	토픽	예
정적 (static)	<code>/tf_static</code>	LiDAR ↔ 선체 (센서는 볼트로 고정됨)
동적 (dynamic)	<code>/tf</code>	선체 ↔ 지도 (배가 움직임)

명령어

```

ros2 run tf2_tools view_frames      # 프레임 트리를 PDF로 저장
ros2 run tf2_ros tf2_echo 상위 하위 # 두 프레임 사이 변환을 실시간 출력

```

★ `frame_id` 가 부정확하면 TF2 변환이 성립하지 않는다

2주차에 배운 `header.frame_id` 가 여기서 값을 함.

2부 · 실습

2-1. Gazebo Garden 설치

저장소 등록

```
sudo apt update
sudo apt install -y lsb-release curl gnupg
```

```
sudo curl https://packages.osrfoundation.org/gazebo.gpg \
  --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
```

⚠ 위 명령은 한 줄이다. 회색 상자 전체를 복사한다.

설치

```
sudo apt update
sudo apt install -y gz-garden
```

ROS 2 연동 패키지

```
sudo apt install -y python3-sdformat13 ros-humble-ros-gzgzarden ros-humble-xacro
```

설치 확인

```
gz sim -v 4 shapes.sdf
```

- **정상:** 도형(구·상자·원기둥)이 놓인 3D 창이 뜸
- 처음 실행 시 모델 다운로드로 시간이 걸릴 수 있음
- 창이 안 뜨면 → 1주차 **xeyes** 확인으로 복귀
- 종료: 창을 닫거나 터미널에서 **Ctrl + C**

2-2. VRX 설치

1단계 — 폴더 만들고 내려받기

```
mkdir -p ~/vrx_ws/src
cd ~/vrx_ws/src
git clone https://github.com/osrf/vrx.git
```

2단계 — 브랜치 지정 (가장 중요)

```
cd ~/vrx_ws/src/vrx
git checkout humble
```

✕ 생략할 수 없는 단계

- VRX의 기본 브랜치는 **최신 조합(ROS 2 Jazzy + Gazebo Harmonic)** 을 따라감
- 본 과목에서는 **Humble + Garden** 이므로 브랜치를 바꿔야 함
- 빠뜨리면 빌드가 실패하거나, 실행할 때 플러그인을 못 찾음
- **매년 가장 많이 나오는 실수임**

- 확인

```
git branch --show-current
```

- 정상 출력: `humble`

3단계 — 의존성 설치

```
cd ~/vrx_ws
rosdep install --from-paths src --ignore-src -r -y
```

4단계 — 빌드

```
source /opt/ros/humble/setup.bash
colcon build --merge-install
```

⚠ 30~60분이 소요된다

- 노트북을 절전 모드로 두지 말 것
- 빌드 도중 프로세스가 죽으면 **메모리 부족임**. 아래로 다시 시도

```
colcon build --merge-install --parallel-workers 2
```

- 정상 완료 출력

```
Summary: 12 packages finished [43min 21s]
```

5단계 — 환경 자동 적용

```
echo "source ~/vrx_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

i 2주차 `capstone_ws` 와 함께 쓰기

`.bashrc` 에 두 줄이 다 있으면 됨. 나중에 적힌 쪽이 우선함.

2-3. 첫 실행 — 배를 물에 띄운다

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

- 시드니 레가타 해역에 WAM-V가 떠 있으면 성공

확인할 것 3가지

확인	의미
파도가 움직이는가	파랑 플러그인 동작 중
배가 파도를 따라 흔들리는가	유체력 · 부력 플러그인 동작 중
창 하단 RTF 값	1.0에 가까울수록 좋음. 0.3 이하면 무거운 것

🔪 창이 표시되기까지 시간이 소요된다

처음 실행하면 모델을 온라인에서 받음. 몇 분 기다릴 것.
받은 모델은 `~/.gz/fuel/` 에 저장되어 다음부터는 빠름.

2-4. 토픽 탐색

- 새 분할에서 실행 (VRX는 켜 둔 채)

```
ros2 topic list
```

- 수십 개가 나옴. 걸러서 봄

```
ros2 topic list | grep sensors  
ros2 topic list | grep thrusters
```

센서 확인

```
ros2 topic echo /wamv/sensors/gps/gps/fix --once
```

```
ros2 topic hz /wamv/sensors/gps/gps/fix
```

```
ros2 topic echo /wamv/sensors/imu/imu/data --once
```

```
ros2 topic hz /wamv/sensors/lidars/lidar_wamv_sensor/points
```

✗ LiDAR 토픽에는 `echo` 를 사용하지 않는다

초당 수십만 개의 점이 글자로 쏟아져 터미널이 마비됨.
`hz`, `bw`, `info` 만 쓸 것.

QoS 확인 — 2주차 복습

```
ros2 topic info /wamv/sensors/imu/imu/data --verbose
```

- `Reliability` 값을 적어 둘 것

- 4주차 과제(토픽 전수조사표)의 한 열이 됨

2-5. 배를 직접 움직여 본다

- 기본 WAM-V는 좌 · 우 2개 추진기를 가짐

직진

- 터미널 A

```
ros2 topic pub --rate 10 /wamv/thrusters/left/thrust std_msgs/msg/Float64 "{data: 200.0}"
```

- 터미널 B

```
ros2 topic pub --rate 10 /wamv/thrusters/right/thrust std_msgs/msg/Float64 "{data: 200.0}"
```

선회 (차동 추진)

- 좌현 약하게, 우현 강하게 → 좌선회

```
ros2 topic pub --rate 10 /wamv/thrusters/left/thrust std_msgs/msg/Float64 "{data: 50.0}"
```

```
ros2 topic pub --rate 10 /wamv/thrusters/right/thrust std_msgs/msg/Float64 "{data: 300.0}"
```

정지

- `data: 0.0` 으로 바꾸거나 `Ctrl + C`

관찰 과제 (필수)

관찰	질문
1	추력을 끊었을 때 얼마나 미끄러지는가?
2	선회 명령에 즉시 도는가, 지연이 있는가?
3	직진 명령만 줬는데 옆으로 흐르는가?

★ 이 세 가지가 왜 배 제어가 어려운지의 전부임

- 급정지가 안 됨 (관성)
- 응답이 느림
- 옆으로 흐름 (sway — 배는 자동차가 아님)

11주차에 충돌회피를 설계할 때 이 관찰을 반드시 기억할 것.

육상 로봇용 회피 알고리즘을 그대로 가져오면 실패하는 이유가 여기 있음.

2-6. TF2 확인

- VRX가 실행 중인 상태에서

```
ros2 run tf2_tools view_frames
```

- 몇 초 기다린 뒤 자동 종료됨
- 현재 폴더에 `frames_*.pdf` 생성 → 열어서 트리 구조 확인

```
ros2 topic echo /tf_static --once
```

- 여기서 **실제 프레임 이름**을 확인한 뒤

```
ros2 run tf2_ros tf2_echo <상위프레임> <하위프레임>
```

RViz2 로 보기

```
rviz2
```

- 좌하단 **Add** → **TF** 추가
- 좌측 **Global Options** → **Fixed Frame** 을 최상위 프레임으로 설정
- Add** → **PointCloud2** → Topic 을 LiDAR 토픽으로 지정
- 배를 움직이면서 프레임과 점군이 함께 움직이는지 확인

🔪 RViz2 의 부하가 큰 경우

LiDAR 표시는 끄고 TF만 볼 것.

마무리

이번 주차 요약

순서	한 일	확인 방법
1	Gazebo Garden 설치	<code>gz sim shapes.sdf</code>
2	VRX 내려받기 + 브랜치 변경	<code>git branch --show-current</code> → <code>humble</code>
3	VRX 빌드	<code>colcon build --merge-install</code> 완료
4	WAM-V 스폰	시드니 레가타에 배가 떠 있음
5	토픽 조사	GPS / IMU / LiDAR 확인
6	배 조종	직진 · 선회 성공
7	TF 트리 확인	<code>frames_*.pdf</code> 생성

수업 진도 체크

★ 다음 주 수업을 따라가기 위한 최소 조건

이론 이해

- ☐ SDF · URDF · Xacro 의 차이를 설명할 수 있다
- ☐ RTF 가 무엇이고 왜 확인해야 하는지 안다
- ☐ 선박 6자유도의 이름과 뜻을 말할 수 있다
- ☐ 왜 3자유도(Surge · Sway · Yaw)만 제어하는지 안다
- ☐ **ENU 와 NED 의 차이**를 그림으로 설명할 수 있다
- ☐ 좌표계 실수는 **에러가 나지 않는다**는 것을 안다
- ☐ 쿼터니언 순서가 ROS와 MATLAB에서 다르다는 것을 안다
- ☐ TF2 가 무엇을 해결하는지 설명할 수 있다

실습 완료

- ☐ `gz sim shapes.sdf` 실행 성공
- ☐ `git branch --show-current` 가 `humble` 출력 ← 반드시 확인
- ☐ `colcon build --merge-install` 성공
- ☐ `competition.launch.py world:=sydney_regatta` 로 WAM-V 스폰
- ☐ 파도가 움직이고 배가 흔들리는 것을 확인
- ☐ GPS / IMU 토픽을 `echo` , `hz` 로 확인
- ☐ LiDAR 토픽의 QoS `Reliability` 를 확인하고 적어 두었다
- ☐ `ros2 topic pub` 으로 배를 **직진**시켰다
- ☐ `ros2 topic pub` 으로 배를 **선회**시켰다
- ☐ `view_frames` 로 TF 트리 PDF 를 생성했다
- ☐ RViz2 에서 TF 를 표시했다

관찰 기록

- ☐ 추력을 끊은 뒤 미끄러지는 거리를 관찰했다

- ☐ 선회 명령의 응답 지연을 관찰했다
- ☐ 직진 명령에도 옆으로 흐르는 것을 관찰했다

과제 3 — 좌표 변환 노드

- **제출 기한:** 4주차 수업 전
- **제출:** 코드 + 검증 결과 + 짧은 분석

① frame_converter 노드 작성

- 위치: `~/capstone_ws/src/usv_basics/usv_basics/frame_converter.py`

구독

- `/wamv/sensors/gps/gps/fix` (`sensor_msgs/NavSatFix`)
- `/wamv/sensors/imu/imu/data` (`sensor_msgs/Imu`)

처리

1. GPS의 LLA를 **flat-earth** 근사로 NED 위치 (`N, E, D`) [m] 로 변환
 - 기준점 `lla0` 는 파라미터로 받거나, 첫 수신값을 기준으로 삼을 것
2. IMU 쿼터니언 → **오일러각 (roll, pitch, yaw)** [rad]
 - 순서 (`x, y, z, w`) 주의
3. ENU 기준 yaw → **NED 기준 선수각** 으로 변환하고 -180 ~ +180 도로 정리

발행

- `/usv/pose_ned` (`geometry_msgs/PoseStamped` 또는 자유 형식)
- 로그로 `N, E, roll, pitch, yaw_NED` 를 1 Hz 출력

② 검증 — 이 과제의 절반

★ 주장은 수치로 검증한다

검증 1 — 왕복 변환

- LLA → NED → LLA 했을 때 원래 값으로 돌아오는가?
- 오차를 **미터 단위로 보고** (1 mm 이하가 정상)

검증 2 — 방향 확인

조작	확인
배를 북쪽 으로 직진	N 값이 증가하는가? 선수각이 0도에 가까운가?
배를 동쪽 으로 직진	E 값이 증가하는가? 선수각이 +90도에 가까운가?

- 스크린샷 또는 로그로 제출

검증 3 — 각도 정리

- 배를 한 바퀴 이상 선회
- 선수각이 179도 → -180도 로 넘어가는 지점에서 **값이 튀지 않는가?**

③ 분석 (5~10줄)

1. ENU → NED 변환을 **빠뜨렸다면** 배는 어떻게 움직이겠는가? 구체적으로
2. 쿼터니언 순서를 혼동하면 어떤 증상이 나타나는가?
3. flat-earth 근사가 유효한 범위는 어느 정도이며, 넘으면 어떤 오차가 생기는가?

평가 기준

항목	배점
노드 정상 동작 (구독 · 변환 · 발행)	30%
검증 1~3 수행 및 수치 제시	40%
분석의 정확성	20%
코드 가독성 (변환 함수 분리, 좌표계 주석)	10%

i 이번 학기 내내 이 기준으로 평가함

절반은 "동작하는 코드"가 아니라 "동작함을 증명한 기록" 임.

막혔을 때

증상	원인	해결
<code>colcon build</code> 중 프로세스가 죽음	메모리 부족	<code>--parallel-workers 2</code> , <code>.wslconfig</code> 메모리 상향
빌드는 됐는데 실행 시 플러그인 오류	브랜치 미지정	<code>cd ~/vrx_ws/src/vrx && git checkout humble</code> 후 재빌드
Gazebo 창이 안 뜨고 멈춤	WSLg 미작동	<code>wsl --update</code> → <code>xeyes</code> 확인
Gazebo가 매우 느림 (RTF < 0.2)	GPU 미사용 / RAM 부족	다른 프로그램 종료, 워크스테이션 사용
<code>ros2 topic list</code> 에 wamv 토픽이 없음	환경 미적용	<code>source ~/vrx_ws/install/setup.bash</code>
<code>ros2 topic pub</code> 했는데 배가 안 움직임	토픽명 불일치	<code>ros2 topic list grep thrusters</code> 로 정확한 이름 확인
<code>view_frames</code> 가 빈 PDF 생성	TF 발행 노드 미실행	VRX 실행 확인, 몇 초 더 대기
모델 다운로드가 멈춤	네트워크	재시도. 캐시는 <code>~/.gz/fuel/</code>

참고 자료

공식 문서

- Gazebo Garden — <https://gazebosim.org/docs/garden>
- VRX Wiki — <https://github.com/osrf/vrx/wiki>
- VRX 시스템 준비 — https://github.com/osrf/vrx/wiki/preparing_system_tutorial
- ROS REP-103 (좌표계 표준) — <https://www.ros.org/reps/rep-0103.html>
- tf2 튜토리얼 — <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>
- URDF 튜토리얼 — <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

볼트 내 문서

- [ENU와-NED를-섞으면-조용히-틀린다](#) — 이번 주차 이론의 핵심
- [배는-급정지하지-않는다](#) — 2-5 관찰 과제의 배경
- [WSL-VRX-환경구축 §4~§5](#) — 설치 절차
- [VRX-월드와-패키지](#) — 월드 목록과 토픽 정리

강의자료 폴더

- **2_지난학기_강의자료/2025/11_동역학모델.pdf** — NED / Body 프레임, 운동학과 동역학의 구분. 이번 주차 1-4 ~ 1-6의 이론적 배경. 반드시 읽을 것

교재

- Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. — Ch. 2 (Kinematics)
- Fossen, *Lecture Notes TTK4190* (NTNU, 무료 공개)
- MSS Toolbox — <https://github.com/cybergalactic/MSS>

다음 주 예고

- 4주차 — VRX 심화: 토픽 전수조사, WAM-V 모델 구조, Mapviz
- 할 일
 - WAM-V의 URDF를 뜯어보고 센서를 직접 추가 · 이동
 - 위성지도 위에 항적을 그리는 Mapviz 설정
 - 팀 공용 문서가 될 토픽 전수조사표 완성
- 준비물
 - 이번 주차 완성한 VRX 환경
 - **git checkout humble** 을 확인하지 않은 학생은 반드시 먼저 확인